

Runtime 3D Placement Editor

Rfice MyRoomEditor 기반 런타임 배치/편집 시스템 | 김철규

이 문서는 Rfice MyRoomEditor 개발 경험을 기반으로 정리한 공개용 코드 샘플입니다.
게임의 하우스링 시스템, 맵 에디터, 런타임 오브젝트 배치 시스템과 유사한 3D 상호작용 구조를 보여주기 위해 회사 원본 코드를 일반화했습니다.
Runtime Editor``Raycast``Placement Validation
State Pattern``Unity C#

Pointer Input

Raycast

Placement Check

Edit State

Apply/Cancel

샘플 목표

- 포인터 기반 3D 오브젝트 선택/배치/이동
- UI 위 클릭과 월드 입력 충돌 방지
- 우선순위 레이어와 무시 오브젝트 필터링
- 이동/회전/색상 편집 기능을 인터페이스로 분리
- 편집 취소 시 이전 상태 복원

설계 기준

- **PlacementInputRaycaster**: RaycastNonAlloc 기반 포인터 감지
- **IPlaceableArea**: 배치 가능한 영역 추상화
- **IEditableProp**: 편집 가능한 오브젝트 계약
- **EditorState**: 배치/선택/이동/회전 상태 분리
- **EditSession**: 적용/취소/복원 정보 관리

원 프로젝트	Rfice - MyRoomEditor
담당 영역	런타임 3D 공간 배치/선택/편집 시스템
샘플 범위	Raycast 입력, UI 차단, 배치 영역 검증, 편집 인터페이스, 상태 전환, 취소 복원
게임 연결성	하우스링 시스템, 맵 에디터, 샌드박스 배치 시스템, 인게임 커스터마이징

편집 가능한 오브젝트 계약

```
public interface IEditableProp
{
    string Id { get; }
    Transform Transform { get; }
    void Focus();
    void Unfocus();
    void Select();
    void Deselect();
    PlacementSnapshot Capture();
    void Restore(PlacementSnapshot snapshot);
    bool TryGetMoveable(out IMoveableProp moveable);
    bool TryGetRotatable(out IRotatableProp rotatable);
    bool TryGetColorEditable(out IColorEditableProp colorEditable);
}

public readonly struct PlacementSnapshot
{
    public Vector3 Position { get; init; }
    public Quaternion Rotation { get; init; }
    public Transform Parent { get; init; }
}
```

포인트

- 오브젝트 공통 편집 계약 정의
- 선택/포커스 시각 상태를 객체 내부에서 처리
- 편집 전 상태를 스냅샷으로 저장해 취소 복원 가능

실무 의도

배치 가능한 오브젝트마다 내부 구조가 달라도 편집 시스템은 IEditableProp만 바라봅니다. 오브젝트 종류가 늘어나도 편집 매니저의 수정 범위를 줄입니다.

이동/회전/색상 기능 분리

```
public interface IMoveableProp
{
    Vector3 Position { get; }
    void Move(Vector3 position);
    bool CanPlaceAt(Vector3 position, IPlaceableArea area);
}

public interface IRotatableProp
{
    Quaternion Rotation { get; }
    void Rotate(Quaternion rotation);
    void RotateBySnap(float angle);
}

public interface IColorEditableProp
{
    IReadOnlyList<Color> AvailableColors { get; }
    void ApplyColor(int index);
}

public interface IPlaceableArea
{
    PlacementType Type { get; }
    Quaternion PlacementRotation { get; }
    bool IsChildAreaOf(out IEditableProp parent);
}

public enum PlacementType
{
    Floor,
    Wall,
    Ceiling,
    PropSurface
}
```

리뷰 포인트

상속으로 모든 오브젝트 기능을 표현하면 클래스 계층이 커집니다. 이 샘플은 기능 인터페이스를 조합해 “움직일 수 있는가”, “회전 가능한가”, “색상 변경 가능한가”를 런타임에 판단합니다.

포인터 기반 Raycast와 우선순위 선택

```
public sealed class PlacementInputRaycaster : MonoBehaviour
{
    [SerializeField] private Camera raycastCamera;
    [SerializeField] private float maxDistance = 50f;
    [SerializeField] private float minDistance = 0.5f;

    private readonly RaycastHit[] _hits = new RaycastHit[10];

    public bool TryGetHit(
        Vector2 screenPosition,
        LayerMask targetMask,
        int priorityLayer,
        out RaycastHit result)
    {
        var ray = raycastCamera.ScreenPointToRay(screenPosition);
        var count = Physics.RaycastNonAlloc(ray, _hits, maxDistance, targetMask);

        if (count == 0)
        {
            result = default;
            return false;
        }

        var validHits = _hits
            .Take(count)
            .Where(hit => hit.collider != null && hit.distance > minDistance)
            .OrderBy(hit => hit.distance)
            .ToArray();

        if (validHits.Length == 0)
        {
            result = default;
            return false;
        }

        var priorityHit = validHits.FirstOrDefault(
            hit => hit.transform.gameObject.layer == priorityLayer);

        result = priorityHit.collider != null ? priorityHit : validHits[0];
        return true;
    }
}
```

실무 의도

RaycastNonAlloc으로 반복 입력 중 GC를 줄이고, 우선순위 레이어로 걸친 오브젝트 중 의도한 편집 대상을 안정적으로 선택합니다.

UI 입력과 월드 입력 충돌 방지

```
public sealed class PointerUiGuard
{
    private readonly EventSystem _eventSystem;
    private readonly List<RaycastResult> _results = new();

    public PointerUiGuard(EventSystem eventSystem)
    {
        _eventSystem = eventSystem;
    }

    public bool IsPointerOnUI(Vector2 screenPosition)
    {
        if (_eventSystem == null)
            return false;

        _results.Clear();

        var eventData = new PointerEventData(_eventSystem)
        {
            position = screenPosition
        };

        _eventSystem.RaycastAll(eventData, _results);
        return _results.Count > 0;
    }
}

public sealed class EditorPointerInput
{
    public Vector2 Position => Mouse.current.position.ReadValue();
    public Vector2 Delta => Mouse.current.delta.ReadValue();
    public bool Pressed => Mouse.current.leftButton.wasPressedThisFrame;
    public bool Released => Mouse.current.leftButton.wasReleasedThisFrame;
    public bool Cancelled => Keyboard.current.escapeKey.wasPressedThisFrame;
}
```

리뷰 포인트

인게임 에디터는 UI 버튼 클릭과 월드 오브젝트 클릭이 동시에 발생하기 쉽습니다. UI 가드를 별도 클래스로 두면 선택/이동/배치 상태에서 같은 정책을 재사용할 수 있습니다.

배치 영역 검증과 자기 자신 무시

```
public sealed class PlacementAreaResolver
{
    private readonly PlacementInputRaycaster _raycaster;
    private readonly PointerUiGuard _uiGuard;
    private readonly LayerMask _areaMask;

    public bool TryGetPlacementArea(
        Vector2 screenPosition,
        IEditableProp ignoredProp,
        out IPlaceableArea area)
    {
        area = null;

        if (_uiGuard.IsPointerOnUI(screenPosition))
            return false;

        if (!_raycaster.TryGetHit(screenPosition, _areaMask, -1, out var hit))
            return false;

        if (!hit.collider.TryGetComponent(out IPlaceableArea candidate))
            return false;

        if (candidate.IsChildAreaOf(out var parent) && parent == ignoredProp)
            return false;

        area = candidate;
        return true;
    }
}
```

문제

이동 중인 오브젝트가 자기 자신의 배치 영역을 다시 감지하면 배치 검증이 흔들릴 수 있습니다.

해결

현재 편집 중인 오브젝트를 필터링하고, UI 위 포인터 입력을 먼저 차단해 월드 조작과 UI 조작이 충돌하지 않게 했습니다.

편집 세션과 취소 복원

```
public sealed class PropEditSession
{
    public IEditableProp Target { get; }
    public PlacementSnapshot Snapshot { get; }
    public bool IsActive { get; private set; }

    public PropEditSession(IEditableProp target)
    {
        Target = target;
        Snapshot = target.Capture();
        IsActive = true;
        target.Select();
    }

    public void Apply()
    {
        if (!IsActive)
            return;

        IsActive = false;
        Target.Deselect();
    }

    public void Cancel()
    {
        if (!IsActive)
            return;

        Target.Restore(Snapshot);
        IsActive = false;
        Target.Deselect();
    }
}

public abstract class EditorState
{
    public abstract void Enter(PropEditSession session);
    public abstract void Tick(EditorPointerInput input);
    public abstract void Exit();
}
```

실무 의도

편집 상태가 많아질수록 취소/적용 정책이 흩어지기 쉽습니다. 세션 객체가 편집 시작 시점의 스냅샷과 적용/취소를 관리하면 상태 클래스는 각자의 조작 로직에 집중할 수 있습니다.

오브젝트 이동 상태

```
public sealed class MovePropState : EditorState
{
    private readonly PlacementAreaResolver _areaResolver;
    private PropEditSession _session;
    private IMoveableProp _moveable;

    public override void Enter(PropEditSession session)
    {
        _session = session;

        if (!session.Target.TryGetMoveable(out _moveable))
            session.Cancel();
    }

    public override void Tick(EditorPointerInput input)
    {
        if (input.Cancelled)
        {
            _session.Cancel();
            return;
        }

        if (!_areaResolver.TryGetPlacementArea(
            input.Position, _session.Target, out var area))
            return;

        var nextPosition = CalculatePlacementPosition(area);
        if (_moveable.CanPlaceAt(nextPosition, area))
            _moveable.Move(nextPosition);

        if (input.Released)
            _session.Apply();
    }

    public override void Exit()
    {
        _session = null;
        _moveable = null;
    }
}
```

포인트

- 편집 가능 여부를 상태 진입 시 검증
- 배치 가능할 때만 위치 적용
- ESC 입력으로 스냅샷 복원

게임 연결성

하우징 가구 이동, 타일/프롭 배치, 샌드박스 오브젝트 편집에 그대로 응용 가능한 구조입니다.

회전 상태와 편집 UI 이벤트 연결

```
public sealed class RotatePropState : EditorState
{
    private PropEditSession _session;
    private IRotatableProp _rotatable;
    private float _accumulatedAngle;

    public override void Enter(PropEditSession session)
    {
        _session = session;
        if (!session.Target.TryGetRotatable(out _rotatable))
            session.Cancel();
    }

    public override void Tick(EditorPointerInput input)
    {
        if (input.Canceled)
        {
            _session.Cancel();
            return;
        }

        _accumulatedAngle += input.Delta.x * 0.35f;
        _rotatable.Rotate(Quaternion.Euler(0f, _accumulatedAngle, 0f));

        if (input.Released)
            _session.Apply();
    }

    public override void Exit()
    {
        _session = null;
        _rotatable = null;
        _accumulatedAngle = 0f;
    }
}

public sealed class ObjectEditMenu : MonoBehaviour
{
    public event Action MoveClicked;
    public event Action RotateClicked;
    public event Action DeleteClicked;
    public event Action ColorClicked;

    public void Show(IEditableProp prop)
    {
        MoveButton.SetActive(prop.TryGetMoveable(out _));
        RotateButton.SetActive(prop.TryGetRotatable(out _));
        ColorButton.SetActive(prop.TryGetColorEditable(out _));
    }
}
```

리뷰 포인트

편집 UI는 선택된 오브젝트의 기능 인터페이스를 확인해 가능한 메뉴만 노출합니다. UI가 모든 오브젝트 타입을 직접 알 필요가 없습니다.

테스트 가능성과 면접 어필 포인트

테스트 가능한 항목

- UI 위 포인터 입력 차단
- 우선순위 레이어 선택
- 자기 자신 배치 영역 무시
- 배치 실패 시 위치 미적용
- 취소 시 이전 위치/회전/부모 복원
- 기능 인터페이스에 따른 UI 메뉴 노출

면접에서 설명할 내용

- 런타임 3D 에디터 구조 경험
- Raycast 입력과 UI 입력 충돌 처리
- State 기반 편집 플로우
- 기능 인터페이스 조합으로 오브젝트 타입 확장
- 하우스징/맵에디터/커스터마이징 시스템으로 확장 가능

포트폴리오 연결 문구

이 샘플은 Rfice MyRoomEditor의 하우스징 시스템 개발 경험을 기반으로 구성했습니다. 실제 프로젝트에서는 3D 공간 배치, 선택, 이동, 회전, 색상 변경, 이미지 적용, 배치 검증 기능을 담당했습니다.

연락처

Portfolio: <https://cosmicgiantkoala.github.io> Email: cosmicgiantkoala@gmail.com